

Applied Machine Learning for Industry Solutions

UNIT 1 – All Home Assignment Questions with 5-Mark Answers

Prepared for exam revision | Simple language | Each answer written according to the question

Preparation basis: The answers are grounded primarily in the Unit 1 question bank and the Unit 1 study material. Where a question itself asks for a numerical metric, Python implementation, threshold effect, or calculation that is not fully specified in the study material, only the minimum necessary calculation or implementation has been added so that the question can be answered.

Important for the exam: Read the question carefully. For a 5-mark answer, write the formula, substitute values where required, show the calculation, and state the final answer clearly.

Q1. For each of Supervised, Unsupervised, and Reinforcement Learning, provide a practical example and explain the differences in data requirements and learning approach.

Answer:

Supervised Learning: In supervised learning, input data and the correct output are given. The algorithm learns a mapping from input X to output Y , written as $Y = f(X)$. The correct answers are known during training. A practical example is predicting a patient's recovery time from patient information.

Unsupervised Learning: In unsupervised learning, the data is unlabeled. The algorithm tries to find hidden patterns or structures in the data. A practical example is grouping patients with similar symptoms.

Reinforcement Learning: In reinforcement learning, an agent interacts with an environment and receives rewards or penalties for its actions. The goal is to learn actions that accumulate maximum reward. A practical example is training a surgical robot.

Difference: Supervised learning requires labeled input-output data and learns from known answers. Unsupervised learning requires unlabeled data and finds patterns or groups. Reinforcement learning learns through interaction with an environment using rewards and penalties.

Q2. Given a probability distribution of events, compute the conditional probability using Bayes' theorem and relate it to a spam email classification example.

Answer:

Bayes' theorem:

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

Where $P(A|B)$ is the conditional probability of event A when event B has occurred. $P(A)$ is the probability of A, $P(B)$ is the probability of B, and $P(B|A)$ is the probability of B when A has occurred.

Spam email example: Let A be the event that an email is spam and B be the event that the email contains a particular word. Then Bayes' theorem can be used to calculate $P(\text{Spam} | \text{Word})$, which is the probability that an email is spam when that word is present.

The Unit 1 material connects Bayes' theorem with Naive Bayes and probabilistic reasoning. Since the question does not provide numerical probability values, a numerical conditional probability cannot be calculated.

Q3. Min-max normalization of price value 7,000 when minimum = 3,000, maximum = 5,000 and new range = [0.5, 2.0].

Answer:

Formula:

$$D = [(d - M_p) / (X_p - M_p)] \times (nX_p - nM_p) + nM_p$$

Substituting the given values:

$$D = [(7000 - 3000) / (5000 - 3000)] \times (2.0 - 0.5) + 0.5$$

$$D = (4000 / 2000) \times 1.5 + 0.5$$

D = 3.5

The normalized value is 3.5. It is outside the required range [0.5, 2.0]. Therefore, an **out-of-bound error** is encountered because 7,000 is greater than the actual maximum value 5,000.

Python Implementation:

```
# Min-Max Normalization
d = 7000
min_value = 3000
max_value = 5000
new_min = 0.5
new_max = 2.0

D = ((d - min_value) / (max_value - min_value)) * (new_max - new_min) + new_min

print("Normalized value =", D)

if D < new_min or D > new_max:
    print("Out-of-bound error")
else:
    print("Value is within the specified range")
```

Q4. Normalize age value 35 using Min-Max, Z-score and Decimal Scaling.

Answer:

Given: Minimum = 13, Maximum = 70, Mean = 29.66, Standard Deviation = 12.94.

(i) Min-Max Normalization

$$D = [(35 - 13) / (70 - 13)] \times (1.0 - 0.0) + 0.0$$

$$D = 22 / 57 = \mathbf{0.38}$$

(ii) Z-score Normalization

$$D = (d - \text{Mean}) / \text{Standard Deviation}$$

$$D = (35 - 29.66) / 12.94 = \mathbf{0.38}$$

(iii) Decimal Scaling Normalization

The value 35 is divided by 100 so that the normalized value is less than 1.

$$D = 35 / 100 = \mathbf{0.35}$$

Final answers: Min-Max = 0.38, Z-score = 0.38, Decimal Scaling = 0.35.

Q5. Apply Simple Linear Regression for the given driving hours and risk score. Find the best-fit equation and predict the risk score for driving hours = 9.

Answer:

Given:

$x = (10, 9, 2, 15, 10, 16, 11, 16)$

$y = (95, 80, 10, 50, 45, 98, 38, 93)$

The simple linear regression equation is:

$$\mathbf{\hat{y} = b_0 + b_1x}$$

The regression coefficients are calculated as:

$$\mathbf{b_1 = \frac{\sum[(x_i - \bar{x})(y_i - \bar{y})]}{\sum[(x_i - \bar{x})^2]}}$$

$$\mathbf{b_0 = \bar{y} - b_1\bar{x}}$$

For the given data:

$\bar{x} = 11.125$ and $\bar{y} = 63.625$

$b_1 = 4.5879$ and $b_0 = 12.5846$

Therefore, the best-fit equation is:

$$\mathbf{\hat{y} = 12.5846 + 4.5879x}$$

For $x = 9$:

$$\hat{y} = 12.5846 + 4.5879(9)$$

$$\hat{y} = \mathbf{53.88}$$

Therefore, the predicted risk score for 9 hours of driving is approximately **53.88**.

Q6. Implement a linear regression model to predict house prices from synthetic data: size, location, rooms, amenities, price; interpret the slope and intercept values.

Answer:

Linear Regression predicts a dependent variable using one or more independent variables. Here, **price** is the dependent variable, while size, location, rooms and amenities are input features.

The implementation below uses sample data and fits a Linear Regression model.

Python Implementation:

```
from sklearn.linear_model import LinearRegression
import numpy as np

# size, location, rooms, amenities
X = np.array([
    [1000, 1, 2, 1],
    [1200, 1, 3, 1],
    [1500, 2, 3, 2],
    [1800, 2, 4, 2]
])

# house price
y = np.array([40, 50, 65, 80])

model = LinearRegression()
model.fit(X, y)

print("Slopes =", model.coef_)
print("Intercept =", model.intercept_)
```

Interpretation: Each slope shows how the predicted house price changes when that feature changes by one unit, while the other features are kept constant. The intercept is the predicted price when all input feature values are zero.

Thus, the model uses the relationship between the input features and house price to make predictions.

Q7. Implement Logistic Regression for gender prediction from height and compute the predicted probability for a given input.

Answer:

Logistic Regression is used when the dependent variable is binary. Although it contains the word regression, it is used for classification.

For the gender example, the first class can be male. The model predicts:

P(gender = male | height)

The logistic regression equation is:

$$y = e^{(b_0 + b_1x)} / [1 + e^{(b_0 + b_1x)}]$$

Using the example from the Unit 1 material:

$b_0 = -100$, $b_1 = 0.6$ and height $x = 150$ cm.

Substituting:

$$y = \exp(-100 + 0.6 \times 150) / [1 + \exp(-100 + 0.6 \times 150)]$$

$$y = 0.0000453978687$$

Therefore, the predicted probability is near zero for the first class in this example.

Python Implementation:

```
import math

b0 = -100
b1 = 0.6
height = 150

z = b0 + b1 * height
p = math.exp(z) / (1 + math.exp(z))

print("Predicted probability =", p)
```


Q8. Explain the working principle of Support Vector Machine (SVM), including hyperplane, margin, support vectors and kernel functions with an example.

Answer:

Support Vector Machine (SVM) is a supervised learning algorithm used for classification. It tries to find the best decision boundary that separates data points belonging to different classes.

Hyperplane: Many decision boundaries may separate two classes. The best decision boundary selected by SVM is called the hyperplane. In two dimensions, it can be a straight line.

Support Vectors: The closest data points from the different classes to the hyperplane are called support vectors.

Margin: The distance between the support vectors and the hyperplane is called the margin. The goal of SVM is to maximize this margin.

Optimal Hyperplane: The hyperplane with the maximum margin is called the optimal hyperplane.

Kernel Function: When data is not linearly separable by a straight line, a kernel function is used to handle non-linear separation.

Example: Suppose data points belong to green and blue classes. If the classes can be separated by a straight line, Linear SVM is used. If they cannot be separated by a straight line, Non-linear SVM with a suitable kernel is used.

Q9. COVID-19 test: TP = 25, FN = 5, TN = 60, FP = 10. Calculate Accuracy, Precision, Recall and F1-score.

Answer:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

$$= (25 + 60) / 100 = 0.85 = 85\%$$

$$\text{Precision} = TP / (TP + FP)$$

$$= 25 / (25 + 10) = 25/35 = 0.7143 = 71.43\%$$

$$\text{Recall} = TP / (TP + FN)$$

$$= 25 / (25 + 5) = 25/30 = 0.8333 = 83.33\%$$

$$\text{F1-score} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

$$= 2 \times 0.7143 \times 0.8333 / (0.7143 + 0.8333)$$

$$= 0.7692 = 76.92\%$$

Final Answer: Accuracy = 85%, Precision = 71.43%, Recall = 83.33%, F1-score = 76.92%.

Q10. Identify the suitable ML type for: predicting patient recovery time, grouping patients with similar symptoms, and training a surgical robot.

Answer:

1. Predict patient recovery time - Supervised Learning: The expected recovery time is an output value. A model can learn from previous patient data and known recovery times. Since the output is a numerical value, this is a regression-type supervised learning problem.

2. Group patients with similar symptoms - Unsupervised Learning: The aim is to group similar patients without requiring known output labels. This is a clustering problem.

3. Train a surgical robot - Reinforcement Learning: A robot can learn by interacting with its environment and receiving rewards or penalties based on its actions.

Conclusion: Prediction with known output uses Supervised Learning, grouping similar unlabeled data uses Unsupervised Learning, and learning through rewards and penalties uses Reinforcement Learning.

Q11. Compute vector addition and dot product for $a = [4, 3]$ and $b = [2, 5]$, and interpret the results.

Answer:

Given: $a = [4, 3]$, $b = [2, 5]$

Vector Addition:

$$a + b = [4 + 2, 3 + 5] = [6, 8]$$

Dot Product:

$$a \cdot b = (4 \times 2) + (3 \times 5)$$

$$= 8 + 15 = \mathbf{23}$$

Interpretation: Vector addition combines corresponding components of the two vectors. The dot product gives a single scalar value, 23.

Python Implementation:

```
import numpy as np

a = np.array([4, 3])
b = np.array([2, 5])

addition = a + b
dot_product = np.dot(a, b)

print("Vector addition =", addition)
print("Dot product =", dot_product)
```

Q12. For $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$, perform $A \times B$ and transpose.

Answer:

Matrix Multiplication:

$A \times B =$

$\begin{bmatrix} 1 \times 5 + 2 \times 7 & 1 \times 6 + 2 \times 8 \end{bmatrix},$

$\begin{bmatrix} 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{bmatrix}$

$= \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

Therefore, $A \times B = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

Transpose: A transpose is obtained by converting rows into columns.

$A^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$

$B^T = \begin{bmatrix} 5 & 7 \\ 6 & 8 \end{bmatrix}$

Python Implementation:

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("A x B =")
print(np.dot(A, B))

print("Transpose of A =")
print(A.T)

print("Transpose of B =")
print(B.T)
```

Q13. Compute and display the gradient vector of $f(x,y) = x^2 + 2xy + 3y^2$ at a non-trivial point.

Answer:

The gradient is a vector of partial derivatives.

Given:

$$f(x,y) = x^2 + 2xy + 3y^2$$

Partial derivative with respect to x:

$$\partial f / \partial x = 2x + 2y$$

Partial derivative with respect to y:

$$\partial f / \partial y = 2x + 6y$$

Therefore, the gradient vector is:

$$\nabla f(x,y) = [2x + 2y, 2x + 6y]$$

Take the non-trivial point (1, 2):

$$\partial f / \partial x = 2(1) + 2(2) = 6$$

$$\partial f / \partial y = 2(1) + 6(2) = 14$$

Therefore:

$$\nabla f(1,2) = [6, 14]$$

Python Implementation:

```
def gradient(x, y):  
    dx = 2*x + 2*y  
    dy = 2*x + 6*y  
    return [dx, dy]  
  
print("Gradient at (1,2) =", gradient(1, 2))
```

Q14. Using a sample dataset containing noisy data, apply binning and clustering techniques to reduce the impact of noise.

Answer:

Binning: First, sort the data and divide it into bins. Then smooth the values using bin mean, bin median or bin boundaries.

Sample noisy data: [2, 3, 5, 6, 8, 9, 9, 10, 12, 15, 17, 20]

Using bins of four values:

Bin A = [2, 3, 5, 6]

Bin B = [8, 9, 9, 10]

Bin C = [12, 15, 17, 20]

For smoothing by bin mean, each value in a bin is replaced by the mean of that bin. Thus, binning smooths local variation by using nearby values.

Clustering: Similar objects are grouped into clusters. It can also help to detect and remove outliers. A noisy value that is very different from other data points may be identified separately.

Python Implementation:

```
import numpy as np
from sklearn.cluster import KMeans

data = np.array([2, 3, 5, 6, 8, 9, 9, 10, 12, 15, 17, 20]).reshape(-1, 1)

# Binning: equal-frequency bins of size 4
sorted_values = np.sort(data.flatten())
bins = [sorted_values[i:i+4] for i in range(0, len(sorted_values), 4)]

smoothed = []
for b in bins:
    smoothed.extend([b.mean()] * len(b))

print("Binning result =", smoothed)

# Clustering
kmeans = KMeans(n_clusters=3, random_state=0, n_init=10)
labels = kmeans.fit_predict(data)

print("Cluster labels =", labels)
print("Cluster centers =", kmeans.cluster_centers_)
```

Q15. Salary attribute: minimum Rs.20,000, maximum Rs.80,000, mean Rs.45,000, standard deviation Rs.10,000. Find the z-score for Rs.60,000 and explain when z-score is preferred.

Answer:

Z-score formula:

$$D = (d - \text{Mean}) / \text{Standard Deviation}$$

For salary $d = \text{Rs.}60,000$:

$$D = (60,000 - 45,000) / 10,000$$

$$D = 15,000 / 10,000$$

$$D = 1.5$$

Therefore, the z-score normalized value of Rs.60,000 is **1.5**.

According to the Unit 1 material, z-score normalization is generally used when the actual minimum and maximum values are not known or when these values are considered outliers.

Python Implementation:

```
salary = 60000
mean = 45000
std = 10000

z = (salary - mean) / std
print("Z-score normalized value =", z)
```


Q16. Demonstrate Univariate, Bivariate and Multivariate Exploratory Data Analysis (EDA).

Answer:

Exploratory Data Analysis (EDA) is the process of analyzing datasets to summarize their main characteristics, often using visual methods. It helps us understand data before applying a Machine Learning model.

1. Univariate Analysis: It analyzes one variable at a time. Common techniques are mean, median, mode, standard deviation, histogram, boxplot and density plot. Example: checking the distribution of age in a customer dataset.

2. Bivariate Analysis: It examines the relationship between two variables. Numerical-numerical data can use scatter plots and correlation. Categorical-categorical data can use contingency tables and bar plots. Categorical-numerical data can use box plots and group statistics.

3. Multivariate Analysis: It analyzes more than two variables simultaneously to understand complex interactions. Techniques include pair plots, heatmaps of correlation matrices, PCA and t-SNE.

Thus, Univariate means one variable, Bivariate means two variables, and Multivariate means more than two variables.

Q17. Explain the theory behind Logistic Regression, interpret coefficients, and evaluate model performance.

Answer:

Logistic Regression is used when the dependent variable is dichotomous or binary. It is a predictive analysis technique, but it is used for classification.

The model combines input values using coefficients and predicts a binary outcome. Its equation is:

$$y = e^{(b_0 + b_1x)} / [1 + e^{(b_0 + b_1x)}]$$

The output is a probability between 0 and 1. This probability can then be converted into a class using a threshold.

Coefficient interpretation: b_0 is the constant or intercept. b_1 is the coefficient of input x and represents the effect of x on the model output. For multiple inputs, each feature has its own coefficient.

Model performance: The model is trained on the dataset and its predictions are compared with the known class labels. A numerical performance value cannot be calculated here because the question does not provide a specific dataset or prediction results.

Therefore, Logistic Regression explains the relationship between input variables and a binary output and produces probabilities for classification.

Q18. Hours Studied = 2, 4, 6, 8 and Exam Passed = 0, 0, 1, 1. Estimate coefficients and predict for 5 hours.

Answer:

Linear Regression equation:

$$\mathbf{\hat{y} = b_0 + b_1x}$$

For the given data:

$$x = [2, 4, 6, 8]$$

$$y = [0, 0, 1, 1]$$

Using the least-squares method:

$$\mathbf{b_1 = 0.2}$$

$$\mathbf{b_0 = -0.5}$$

Therefore, the regression equation is:

$$\mathbf{\hat{y} = -0.5 + 0.2x}$$

For 5 hours studied:

$$\hat{y} = -0.5 + 0.2(5)$$

$$\hat{y} = -0.5 + 1$$

$$\mathbf{\hat{y} = 0.5}$$

Therefore, the predicted output for 5 hours studied is **0.5**.

Q19. A Logistic Regression model gives $P(y=1|x) = 0.72$. (a) Classify with threshold = 0.7. (b) Explain the effect on recall if the threshold is lowered to 0.5.

Answer:

(a) Classification:

Given probability = 0.72 and threshold = 0.7.

Since 0.72 is greater than or equal to 0.7:

The input is classified as $y = 1$, or the positive class.

(b) Effect on recall:

When the threshold is lowered from 0.7 to 0.5, more observations can be classified as positive.

Because fewer actual positive cases are rejected, recall can increase or remain the same. However, lowering the threshold can also increase the number of false positives.

Therefore, the main expected effect is that recall generally improves because the model becomes less strict about predicting the positive class.

Q20. Quality control test: TP = 15, FN = 5, TN = 70, FP = 10. Calculate Accuracy, Precision, Recall and F1-score.

Answer:

Accuracy = (TP + TN) / Total

= (15 + 70) / 100 = 0.85 = 85%

Precision = TP / (TP + FP)

= 15 / (15 + 10) = 15/25 = 0.60 = 60%

Recall = TP / (TP + FN)

= 15 / (15 + 5) = 15/20 = 0.75 = 75%

F1-score = 2 × Precision × Recall / (Precision + Recall)

= 2 × 0.60 × 0.75 / (0.60 + 0.75)

= 0.6667 = 66.67%

Final Answer: Accuracy = 85%, Precision = 60%, Recall = 75%, F1-score = 66.67%.

**Q21. Genetic disorder diagnostic tool: TP = 40, FN = 10, TN = 135, FP = 15.
Calculate Accuracy, Precision, Recall and F1-score.**

Answer:

$$\text{Total} = 40 + 10 + 135 + 15 = 200$$

$$\text{Accuracy} = (\text{TP} + \text{TN}) / \text{Total}$$

$$= (40 + 135) / 200 = 175/200 = \mathbf{0.875 = 87.5\%}$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$= 40 / (40 + 15) = 40/55 = \mathbf{0.7273 = 72.73\%}$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$= 40 / (40 + 10) = 40/50 = \mathbf{0.80 = 80\%}$$

$$\text{F1-score} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

$$= 2 \times 0.7273 \times 0.80 / (0.7273 + 0.80)$$

$$= \mathbf{0.7619 = 76.19\%}$$

Final Answer: Accuracy = 87.5%, Precision = 72.73%, Recall = 80%, F1-score = 76.19%.